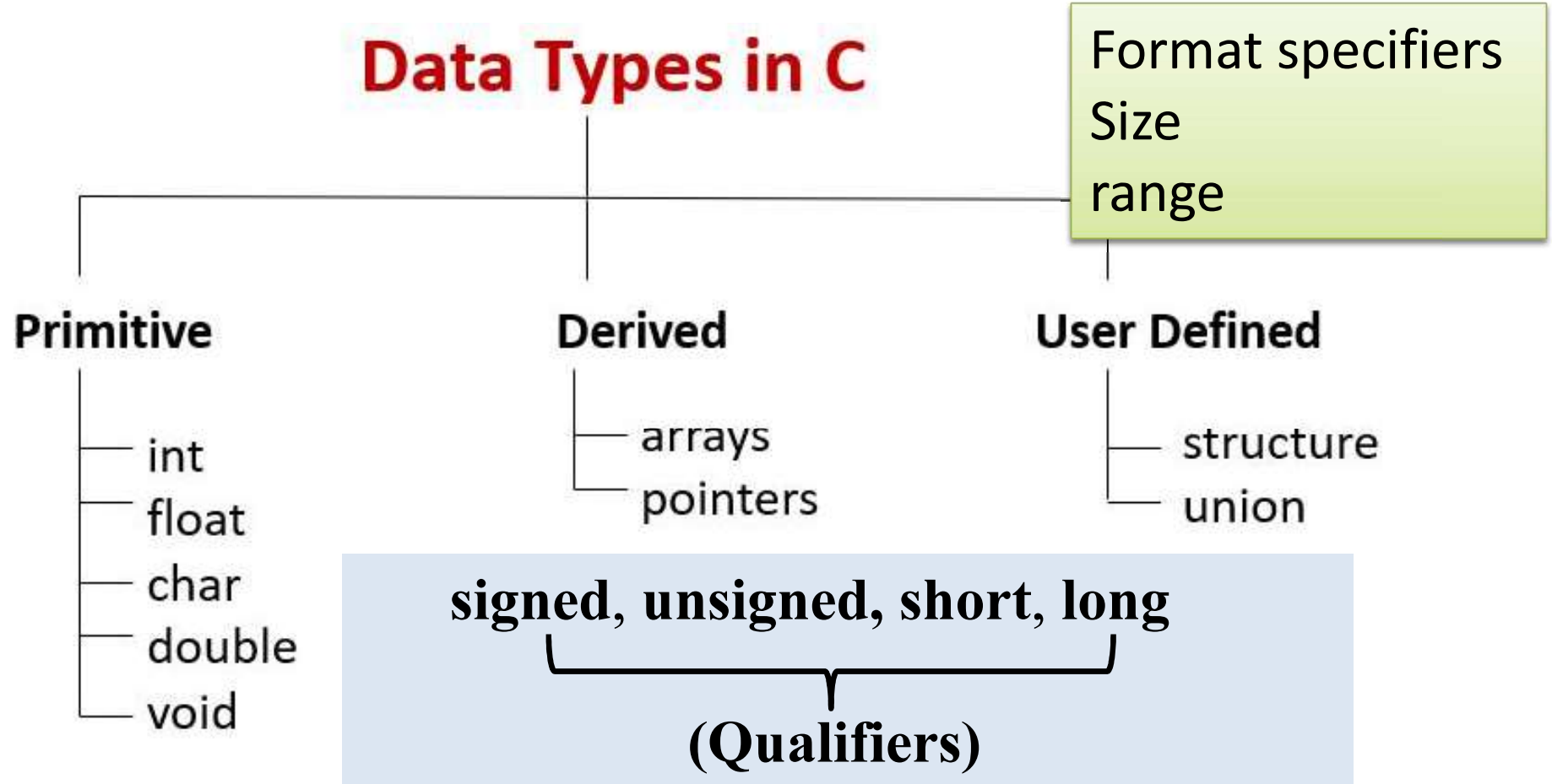


Data Types In C

Data Types in C



- The type **void** either explicitly declares a function as returning no value or creates generic pointers.

Data Types in C

Data type	Size	Range	Format specifier
char	1	-128 to +127	%c
unsigned char	1	0 to 255	%c
unsigned int	4	0 to 4,294,967,295	%u
signed int	4	-2,147,483,648 to +2,147,483,647	%d or %i
short int	2	-32768 to +32767	%hi
long int	4	-2,147,483,648 to +2,147,483,647	%ld
long long	8	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	%lld
float	4	1.17e -38 to 3.4e +38	%f
double	8	2.2e -308 to 1.7 e +308	%lf
Long double	16	3.4e -4932 to 1.18e +4932	%Lf

n bit unsigned range
0 to $2^n - 1$

n bit signed range
 $(- 2^{n-1})$ to $(2^{n-1} - 1)$

Data Types in C

Data Type	Size	Range	Format Specifier
char	1	-128 to +127	%c
unsigned char	1	0 to 255	
unsigned int	2 (for 16 bit compiler)	0 to 65535	%u
	4	0 to 4,294,967,295	
signed int	2 (for 16 bit compiler)	-32768 to +32767	%d or %i
	4	-2,147,483,648 to +2,147,483,647	
short int	2	-32768 to +32767	%hi
long int	4	-2,147,483,648 to +2,147,483,647	%ld
long long	8	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	%lld
float	4	1.17e -38 to 3.4e +38	%f
double	8	2.2e -308 to 1.7 e +308	%lf
Long double	16	3.4e -4932 to 1.18e +4932	%Lf

n bit unsigned range
0 to $2^n - 1$

n bit signed range
(- 2^{n-1}) to ($2^{n-1} - 1$)

Find output of the following program
(Assuming that the program run on a 16-bit system):

```
main()
{
    int a=32768;
    unsigned int b=65536;
    printf("%d,%u",a,b);
}
```

-32768, 0

```
#include<limits.h>
```

```
#include <float.h>
```

```
INT_MAX
```

```
INT_MIN
```

```
UINT_MAX
```

```
SHRT_MIN
```

```
SHRT_MAX
```

```
USHRT_MAX
```

```
LONG_MAX
```

```
LONG_MIN
```

```
ULONG_MAX
```

```
FLT_MAX
```

```
FLT_MIN
```

```
DBL_MAX
```

```
DBL_MIN
```

Type Conversion

Converting one type into another.

- Implicit type conversion
(automatic)
- Explicit type conversion

```
int x=10;  
float a=10.5, b;  
b= a + x;
```

While evaluating **a+x** first x will be converted into float, then addition will be done.

x will automatically be converted into float. This is called **implicit conversion**.

Implicit Type Conversion

```
float x=10.10;  
double a=10.5,  
float y=x *a;
```

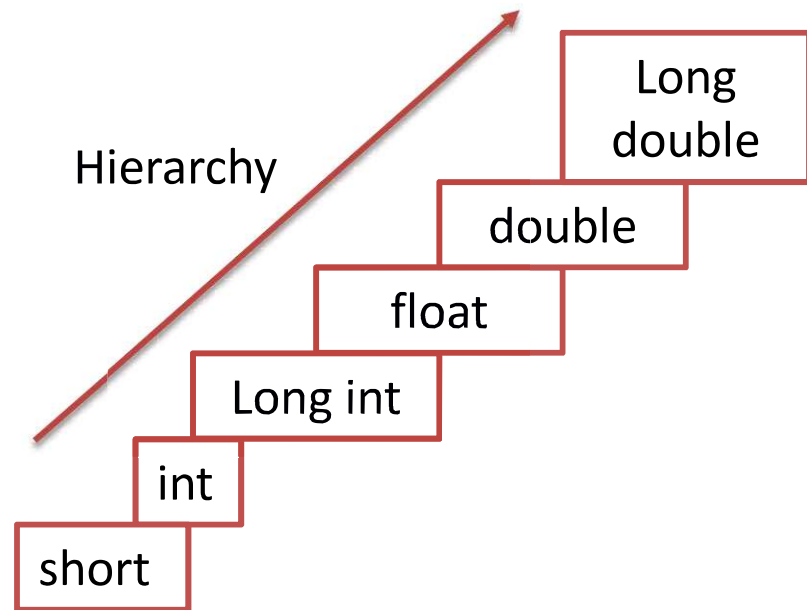
While evaluating $x*a$,
first x will be converted into double,
then multiplication will be done.

Important:

If the operands are of different types-
the lower type is automatically converted
into higher type before the operation
proceeds

Note:

The final result of an expression is
converted to the type of variable on LHS of
assignment operator.



Final result will be float
since y is float

Explicit Type Conversion

Done by programmer

Also known as Type Casting

<pre>float x=10.34,y=45.89; int a; a=(int)x + (int)y ;</pre>	<pre>float x=10.34,y=45.89; int a; a=x +y ;</pre>	<pre>int a=20,b=6; float x; x=(float)a / (float)b;</pre>
--	---	---

Here x and y are forcefully converted into int.

This is called **Explicit Type Conversion**.

Mix-mod operation

➤ $\text{int}/\text{int}=\text{int}$

➤ $\text{int}+\text{int}=\text{int}$

➤ $\text{int}-\text{int}=\text{int}$

➤ $\text{int}*\text{int}=\text{int}$

$\text{int}\%\text{int}=\text{int}$ remainder

$\text{int} \otimes \text{int} = \text{int}$

$\text{float} \otimes \text{int} = \text{float}$

$\text{int} \otimes \text{float} = \text{float}$

$\text{float} \otimes \text{float} = \text{float}$

➤ $\text{float} / \text{int} = \text{float}$

➤ $\text{int} / \text{float} = \text{float}$

➤ $\text{float} / \text{float} = \text{float}$

Operators and Expressions

Operators & Expressions

Precedence (Priority):

- It tells the order in which different operators are evaluated.

for example: In expression $(4+2*3)$

$2*3$ will get evaluated first

Then addition will be done.

Since precedence of $*$ is higher than $+$

Operators & Expressions

Associativity:

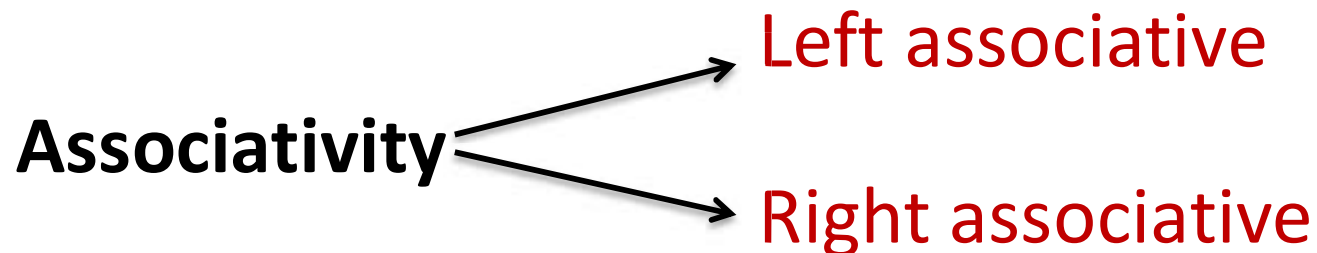
- It tells the order in which operators of *same precedence* will be evaluated.

for example: In expression $(4+2+3)$

$4+2$ will get evaluated first

Then second $+$ will be evaluated.

Since $+$ is **left associative**.



Types of operators

1. Arithmetic operators
2. Relational operators
3. Logical operators
4. Increment/decrement operators
5. Conditional operator
6. Bitwise operators

1. Arithmetic operators

1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
5. Modulus (%)

1. Arithmetic operators

1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
5. Modulus (%)

Precedency:

$*, / , %$

$+, -$

Associativity: left associative

1. Arithmetic operators

Example:

$$a = 2 * 3 + 4 - 5 * 6$$

here * has higher precedence than + and –
so it will be evaluated first

and between two * , left one will be
evaluated first, since it is left associative.

1. Arithmetic operators

Example: $a = 2 * 3 + 4 - 5 * 6$

$$= 2 * 3 + 4 - 5 * 6$$

$$= 6 + 4 - 5 * 6$$

$$= 6 + 4 - 30$$

$$= 10 - 30$$

$$= -20$$

2. Relational Operators

1. greater than (**>**)
2. greater than or equal to (**>=**)
3. less than (**<**)
4. less than or equal to (**<=**)
5. equal to (**==**)
6. not equal to (**!=**)

2. Relational Operators

1. greater than (**>**)
2. greater than or equal to (**>=**)
3. less than (**<**)
4. less than or equal to (**<=**)
5. equal to (**==**)
6. not equal to (**!=**)

Precedence:

<, <=, >, >=
==, !=

Associativity: left associative

2. Relational Operators

- These operators are used to define a condition.
- Generally in if-else statements and loops.
- End result of this operator is True/False

(non-zero/zero)

3. Logical operators

Used to join more than one conditions.

1. AND (&&)
2. OR(||)
3. NOT(!)

3. Logical operators

➤ Logical operators:

logical AND (&&)

logical OR(||)

logical NOT (!)

condition1	condition2	AND(&&)	OR()
T	T	T	T
T	F	F	T
F	T	F	T
F	F	F	F

condition	NOT(!)
T	F
F	T

if(a>b)

.....

else

.....

if(b>=a)

3. Logical operators

Used to join more than one conditions.

1. AND (&&)
2. OR(||)
3. NOT(!)

Precedence:

!

&&

||

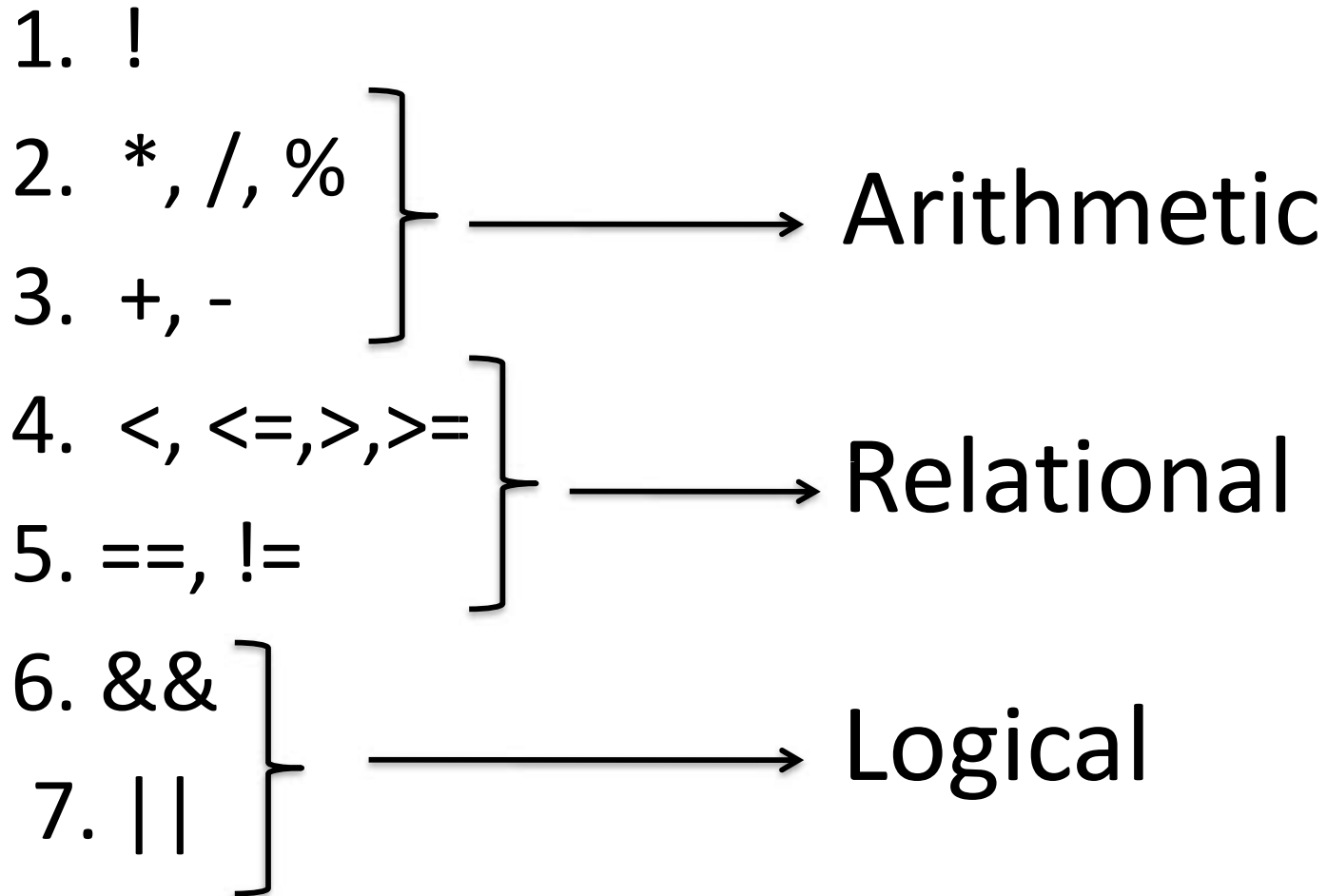
Associativity: left associative

3. Logical operators

Examples:

1. `if(age>55 && salary<10000)`
2. `if(a==b && a!=c)`

Order of Precedence so far



Increment/decrement operators

Increment (**++**)

decrement (**--**)

- ++ adds 1 to the operand
- -- subtracts 1 from the operand

Ex: `int a=5;`

`a++;`

`printf("%d",a);`

Output: 6

Increment/decrement operators

Increment (**++**)

decrement (**--**)

➤ ++ adds 1 to the operand

➤ -- subtracts 1 from the operand

Ex: `int a=5;`

`a--;`

`printf("%d",a);`

Output: 4

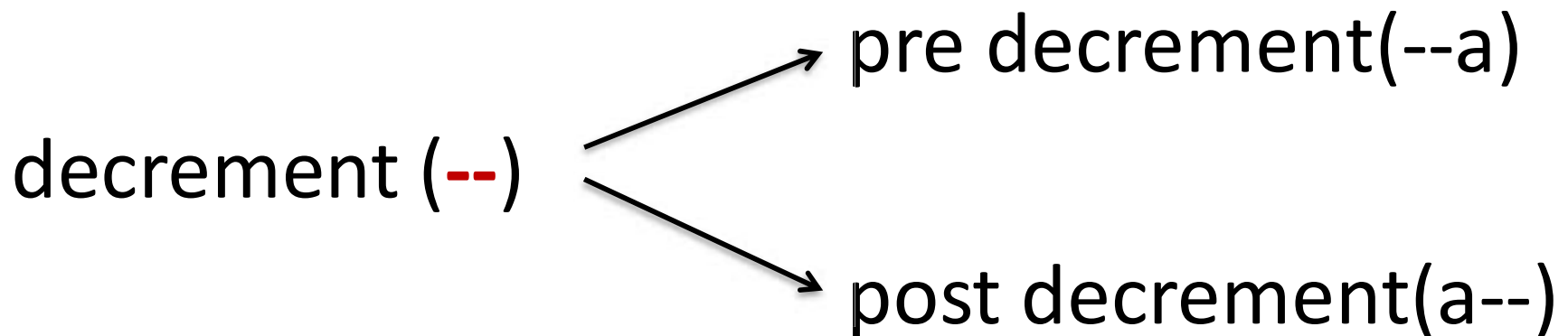
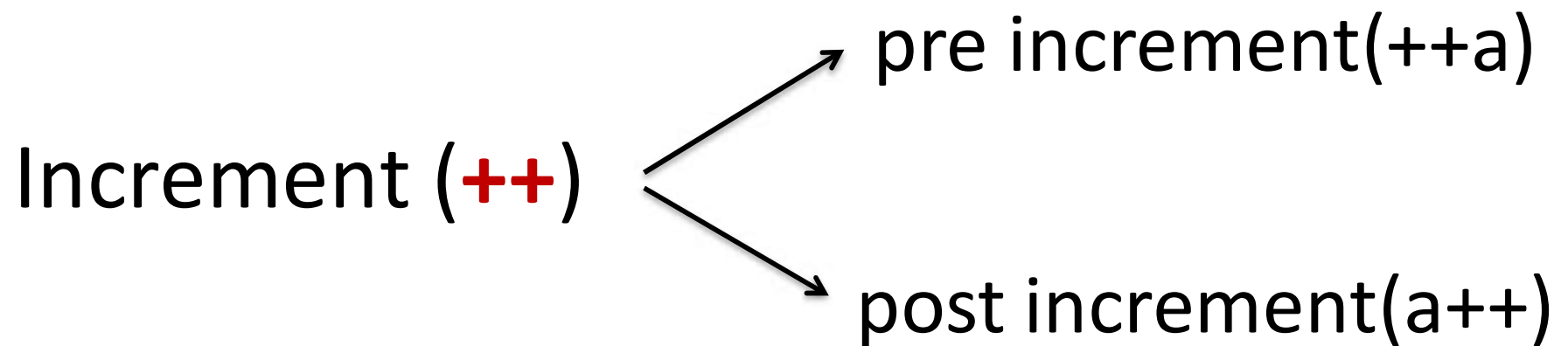
Increment/decrement operators

Increment (**++**)

decrement (**--**)

- `++a` or `a++` means `a=a+1`
- `--a` or `a--` means `a=a-1`

Increment/decrement operators



Increment/decrement operators

Pre increment(++a) :

b=++a;

- first a will be incremented
- then the value of a will be assigned to b.

$$b=++a; \left. \begin{array}{l} a=a+1; \\ b=a; \end{array} \right\}$$

Post increment(a++) :

b=a++;

- first value of a will be assigned to b
- then a will be incremented.

$$b=a++; \left. \begin{array}{l} b=a; \\ a=a+1; \end{array} \right\}$$

Increment/decrement operators

Example:

```
a=5;
```

```
b=a++;
```

```
printf(“%d”,b);
```

output: 5

```
a=5;
```

```
b=++a;
```

```
printf(“%d”,b);
```

output: 6

Increment/decrement operators

Example:

```
a=5;
```

```
b=a++;
```

```
printf("b=%d",b);
```

```
printf(" a=%d",a);
```

output: b=5 a=6

```
a=5;
```

```
b=++a;
```

```
printf("b=%d",b);
```

```
printf(" a=%d",a);
```

output: b=6 a=6

Increment/decrement operators

Pre decrement(--a) :

b= --a;

- first a will be decremented
- then the value of a will be assigned to b.

$$b = --a; \left. \begin{array}{l} a = a - 1; \\ b = a; \end{array} \right\}$$

Post decrement(a--) :

b= a--;

- first value of a will be assigned to b
- then a will be decremented.

$$b = a--; \left. \begin{array}{l} b = a; \\ a = a - 1; \end{array} \right\}$$

```

#include<stdio.h>
int main()
{
    int a=5;
    int b=a++ + a++; // 11
    printf("b=%d",b);

    a=5;
    int c=++a + ++a; // 14
    printf("\nc=%d",c);

    a=5;
    int d=++a + a++; // 13
    printf("\nd=%d",d);

    a=5;
    int e=a++ + ++a; //12
    printf("\ne=%d",d);

    return 0;
}

```

```

#include<stdio.h>
int main()
{
    int a=5;
    printf("a++:%d  a++:%d",a++,a++);

    a=5;
    printf("\n++a:%d  ++a:%d",++a,++a);

    a=5;
    printf("\n++a:%d  a++:%d  ",++a,a++);

    a=5;
    printf("\na++:%d  ++a:%d",a++,++a);

    return 0;
}

```

```

a++:6  a++:5
++a:7  ++a:7
++a:7  a++:5
a++:6  ++a:7

```

Conditional operator

➤ To check condition and take decision.

(? :)

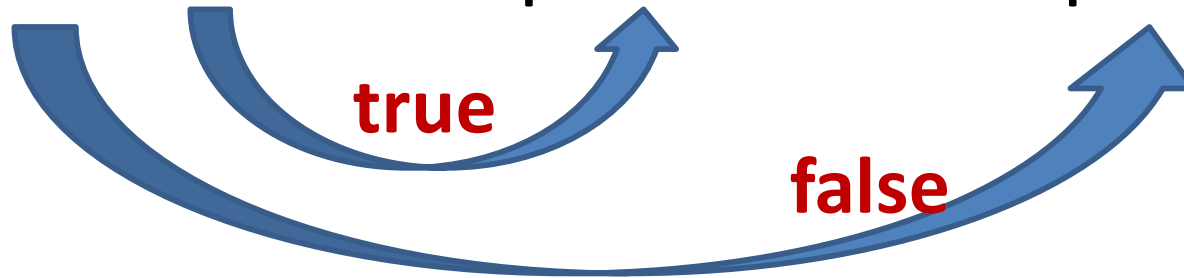
syntax: expression1 ? expression 2 : expression3;

Working:

expression 1 is evaluated first for true or false;
if it is true then expression 2 will be evaluated,
otherwise expression3 will be evaluated.

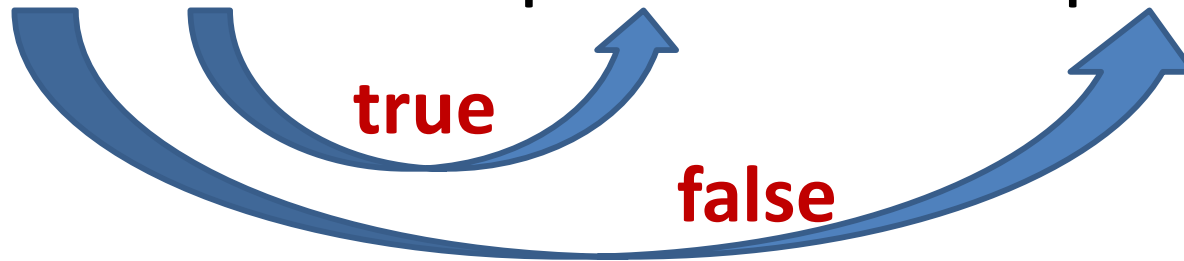
Conditional operator

expression1 ? expression 2 : expression3;



Conditional operator

expression1 ? expression 2 : expression3;



```
if(exprssion1)
    expression2;
else
    expression3;
```

Conditional operator

Greater between two distinct number using conditional operator.

```
#include<stdio.h>
int main()
{
    int a,b,max;
    printf("enter two numbers");
    scanf("%d%d",&a,&b);
    max=a>b?a:b;
    printf("%d",max);
    return 0;
}
```



```
if(a>b)
    max=a;
else
    max=b;
```

Conditional operator

Greater between two distinct number using conditional operator.

```
#include<stdio.h>  
int main()  
{  
    int a,b;  
    printf("enter two numbers");  
    scanf("%d%d",&a,&b);  
    a>b?printf("%d",a) : printf("%d",b);  
  
    return 0;  
}
```

Conditional operator

Greatest between three distinct number using conditional operator.

```
if(a>b)
{
    if(a>c)
        max=a;
    else
        max=c;
}
else
{
    if(b>c)
        max=b;
    else
        max=c;
}
```

max=a>b?(a>c?a:c):(b>c ?b:c)

Conditional operator

Greatest between three distinct number using conditional operator.

```
#include<stdio.h>  
int main()  
{  
    int a,b,c,max;  
    printf("enter three numbers");  
    scanf("%d%d%d",&a,&b,&c);  
    max=a>b?(a>c?a:c):(b>c ?b:c);  
    printf("%d",max);  
    return 0;  
}
```

Bit-wise operators

- Manipulation of data at bit level.
- Will not be applied for float

Types:

1. Bitwise NOT (~)
2. Bitwise AND (&)
3. Bitwise OR (|)
4. Bitwise XOR(^)
5. Shift left(<<)
6. Shift right(>>)

Bitwise operators

int a=10,b=28;

a	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0
b	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0

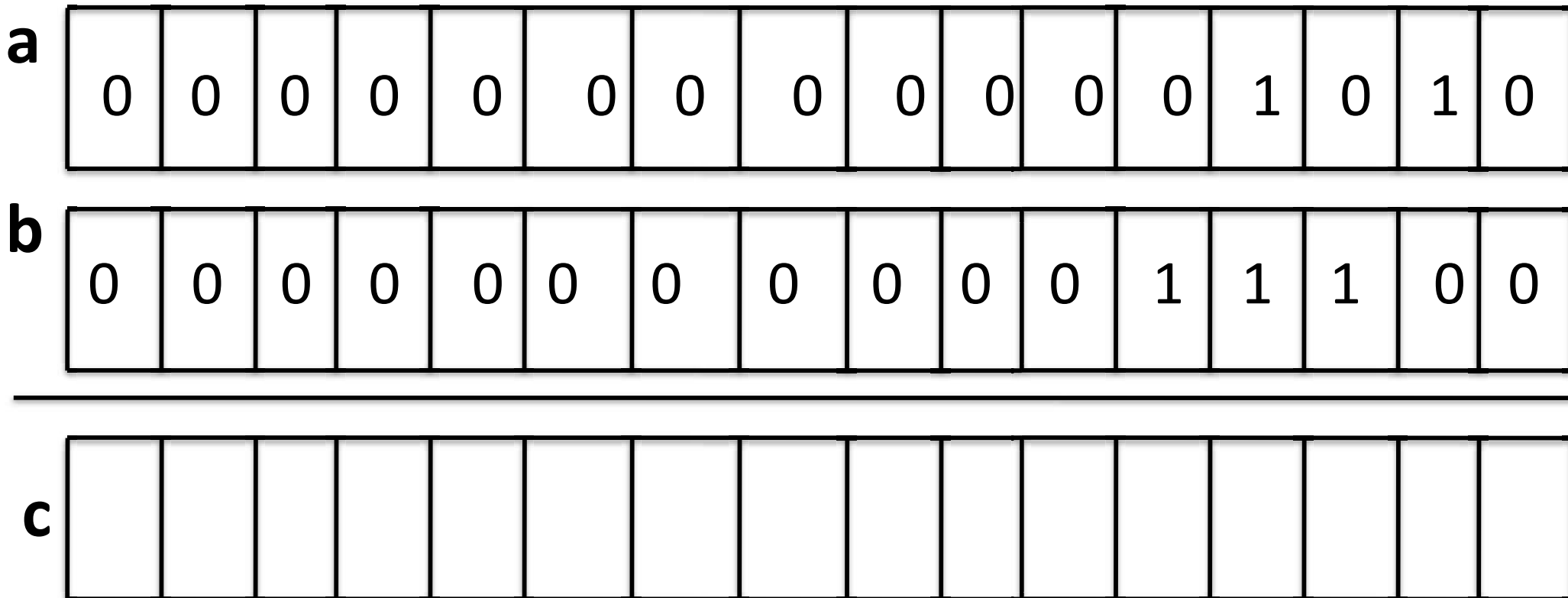
$\sim a = -11$

$\sim b = -29$

Bitwise operators

int a=10,b=28,c;

c=a&b;



Bitwise operators

bitwise AND (&)

bitwise OR(|)

bitwise XOR (^)

op1	op2	&		^
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Bitwise operators

int a=10,b=28,c;

c=a&b;

a	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0
b	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0
c	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0

c=8

Bitwise operators

int a=10,b=28,c;

c=a | b;

a

0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

b

0	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

c

0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

c=30

Bitwise operators

```
int a=10,b=28,c;
```

```
c=a^b;
```

a

0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

b

0	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

c

0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

c=22

Bitwise operators

- shift left (<<): shifting bits towards left
- Shift right (>>): shifting bits towards right

Bitwise operators

left shift(<<):

int a=10,b=28,c;

c=a<<3;

a

0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

0

0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

0 0

0	0	0	0	0	0	0	0	0	0	0	1	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

0 0 0

0	0	0	0	0	0	0	0	0	0	1	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---

c=80

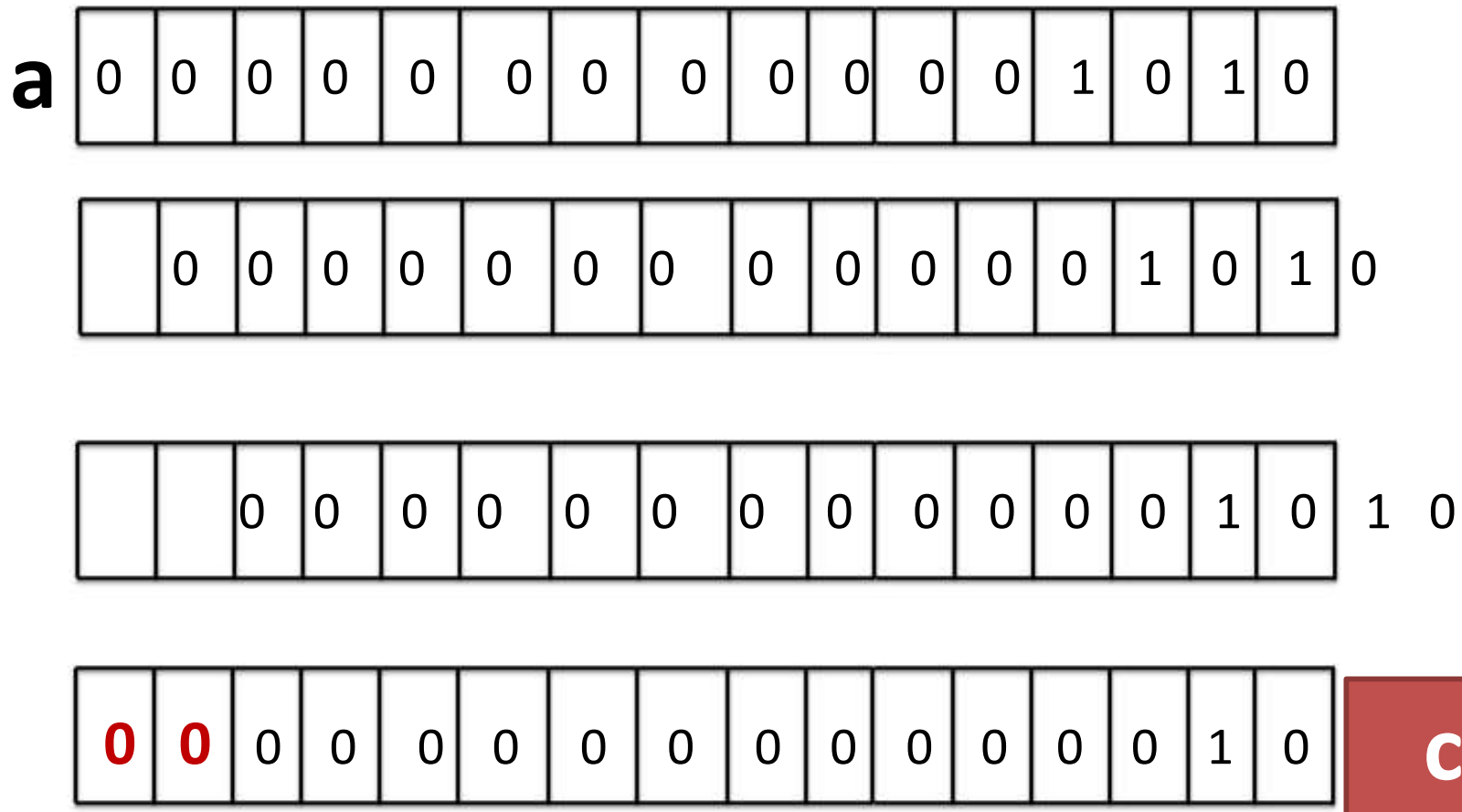
0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Bitwise operators

right shift (>>):

int a=10, c;

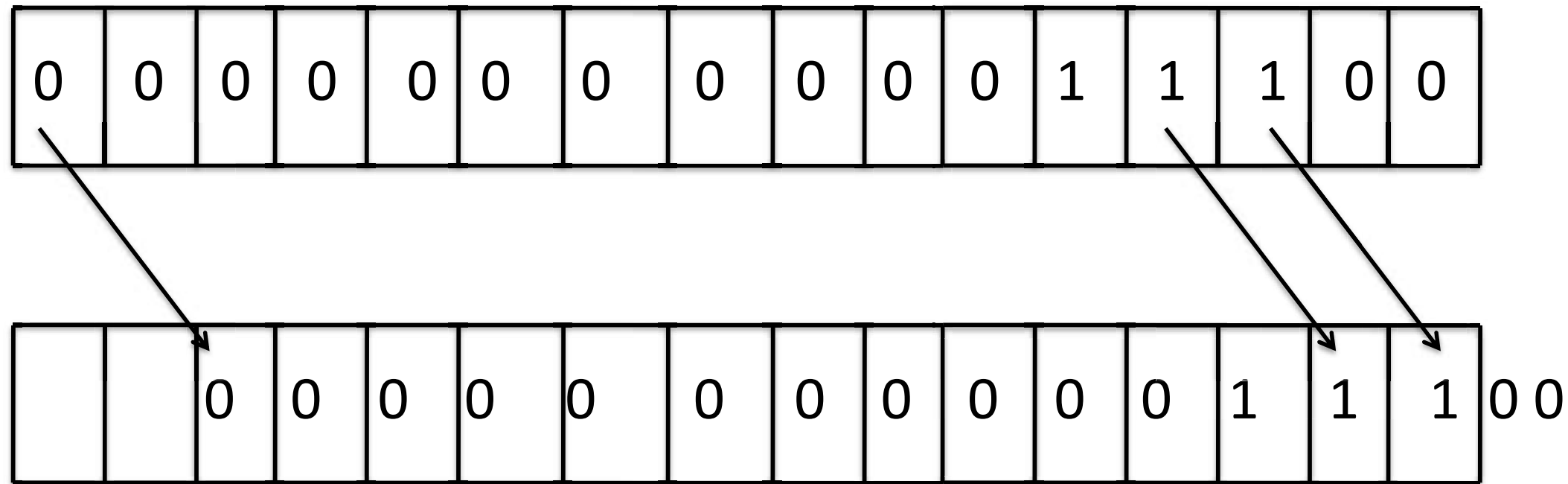
c=a>>2;



Bitwise operators

```
int b=28,d;
```

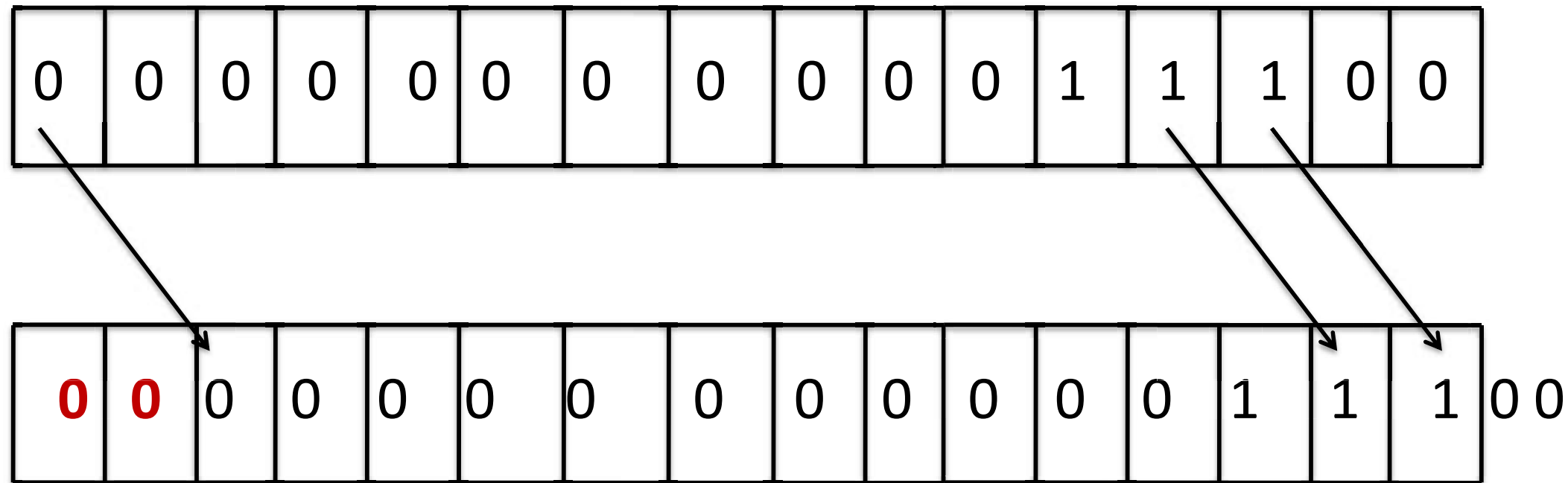
```
d=b>>2
```



Bitwise operators

int b=28,d;

d=b>>2



d=7

Special Operators

- Comma
- Sizeof
- & and *

Special Operators

The comma operator

➤ comma as a separator:

```
int a,b,c;  
int x=10,y=39,z=34;
```

➤ comma as an operator:

```
int a;  
a=(10,20,30);  
printf("a=%d",a); //30
```

- It returns the rightmost operand
- It evaluates all the expressions, but returns the rightmost one
- Least precedence among all operators
- Left associative
- used to link the related expressions together

```
int a;  
a=10,20,30;  
printf("a=%d",a); //10
```

‘=’ has higher precedence than ‘,’
this is why a=10 gets evaluated first

```
x= (a=10, b=20, a+b);  
printf(“%d”,x);    // 30
```

```
int x=19, 30,49; // int x=19, int 30, int 49; so error
```

Special Operators

The sizeof operator:

➤ It returns the size of operand.

```
sizeof(int)  
sizeof(float)
```

```
int main()  
{  
    int x;  
    printf("%d",sizeof(x));  
}
```

Special Operators

& and *

```
int a=10;  
printf("%d",a);  
printf("%u",&a); // address of a  
printf("%d", *(&a)); // value at (&a) 10
```

Short hand assignment operators

`a=a+1`

`a+=1`

`a=a-1`

`a-=1`

`a=a*(n+1)`

`a*= (n+1)`

`a=a%b`

`a%=b`

Difference between == and =

==

- 1.It is known as relational operator.
- 2.It is used to compare between two operands.

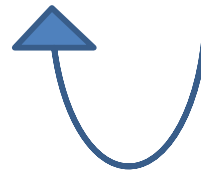
Ex: if(a==b)

It results in true or false

=

- 1.It is known as assignment operator.
- 2.It is used to assign value of right hand side to the variable at left hand side.

Ex: a= 3+4;



Types of operators (According to number of operands)

1. Unary operators (one operand)

Example: !, ++, --, unary -

2. Binary operators (two operands)

+, -, *, /, %, < , >

3. Ternary operators (three operands)

? : conditional operator

OPERATOR	TYPE	ASSOCIATIVITY
() [] . ->		left-to-right
++ -- + - ! ~ (type) * & sizeof	Unary Operator	right-to-left
* / %	Arithmetic Operator	left-to-right
+ -	Arithmetic Operator	left-to-right
<< >>	Shift Operator	left-to-right
< <= > >=	Relational Operator	left-to-right
== !=	Relational Operator	left-to-right
&	Bitwise AND Operator	left-to-right
^	Bitwise EX-OR Operator	left-to-right
 	Bitwise OR Operator	left-to-right
&&	Logical AND Operator	left-to-right
 	Logical OR Operator	left-to-right
? :	Ternary Conditional Operator	right-to-left
= += -= *= /= %= &= ^= = <<= >>=	Assignment Operator	right-to-left
,	Comma	left-to-right